



Agentic Interview Platform

Architecting the Future of Recruitment with MERN,
Multi Agent LLMs and Adaptive Evaluation

Low-latency • Human-in-the-loop • Scalable by design

Why Existing Hiring Platforms Fall Short

Functional Fragmentation



Isolated Tools

Resume screening, coding tests, and interviews live on separate platforms

Static Assessments

No real-world collaboration. Poor signal on communication and teamwork

Bias & Inefficiency



Manual Screening

Keyword-driven, slow, subjective

No Adaptability

Fixed question banks. No real-time difficulty adjustment or personalized feedback

Technical & Usability Limits



Scalability Constraints

Performance degrades beyond ~250 concurrent users. Vertical scaling becomes cost-prohibitive

Network Dependency

Video and code sync degrade on weak connections

SME Market Gap

Enterprise tools too complex and expensive. Affordable tools lack AI ranking and automation

The Foundation: Low-Latency System Architecture



Frontend (Client)

- React.js (SPA)
- Session State Management
- WebRTC (Real-time Video/Audio)



Backend (Orchestration)

- Node.js & Express
- JWT Authentication
- API Gateway



Data & Performance

- MongoDB Atlas (Profiles, Transcripts, Resume Vectors, AI Reports)
- PostgreSQL (Real-time AI Streaming via LISTEN / NOTIFY)
- Redis Caching (Cost Optimization)
- Priority Queue Load Balancing

Advanced Logic: The MockLLM Multi-Agent Framework

Top: Multi-Agent Architecture



Bottom: Continuous Interaction Loop



Hybrid Functionality: Peer-to-Peer Augmentation

AI augments human decision-making — it does not replace it.



Real-Time Media

WebRTC video calls with collaborative code editors and shared whiteboards



Automated Logistics

Cron-based scheduling, time-slot management, reminders, and coordination.



Structured Feedback

Humans and AI use the same scoring frameworks to ensure evaluation consistency.

The Two-Sided "Handshake" Evaluation Protocol

Candidate Signals

Resume Score (Skills Match)

Interview Score (Dialogue Performance)



Role & Interviewer Signals

Job Fit (Candidate Preference)

Role Fit (Interviewer Assessment)

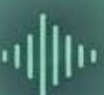
"Match confirmed only upon mutual alignment."

Quantifying Soft Skills



Knowledge Correctness — 40%

Technical accuracy of responses



Fluency — 20%

Smoothness and flow of delivery



Communication — 25%

Clarity and structure of expression



Professionalism — 15%

Tone, confidence, and conduct



AI & Real-Time System Challenges → Architectural Solutions

TOP — Key Challenges



Latency & Scale

Slow LLM responses under concurrency.



Structured Output

JSON inconsistency and schema drift.



Reliability & Trust

Hallucinations and API instability.



Context Limits

Large tasks exceed context windows.



Event Complexity

Async workflows and debugging overhead.

Decompose • Isolate • Coordinate Asynchronously

BOTTOM — Architectural Solutions



Stateful Multi-Agent Architecture

- Specialized agents per interview phase



Async Orchestration (State Machine)

- Centralized correctness and context control



Redis Caching & Streaming Optimization

- Cached prompts and AI outputs
- Reduced latency, herd avoidance



Job Queues + Vector Memory

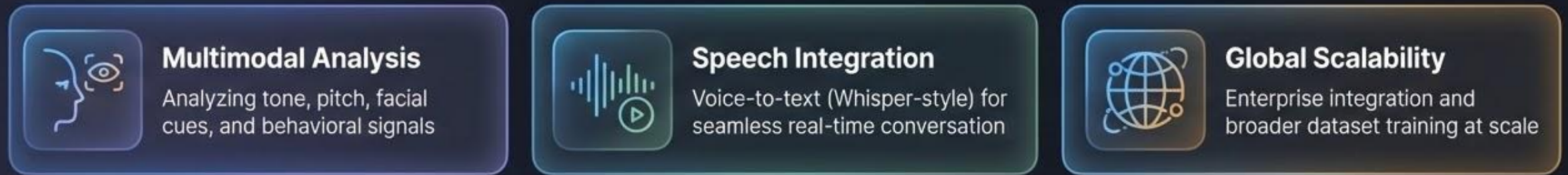
- Async long-running tasks
- Persistent cross-stage context

Engineering for Scale & Performance

Performance & Cost Optimization Pipeline



Scalability & Expansion Capabilities



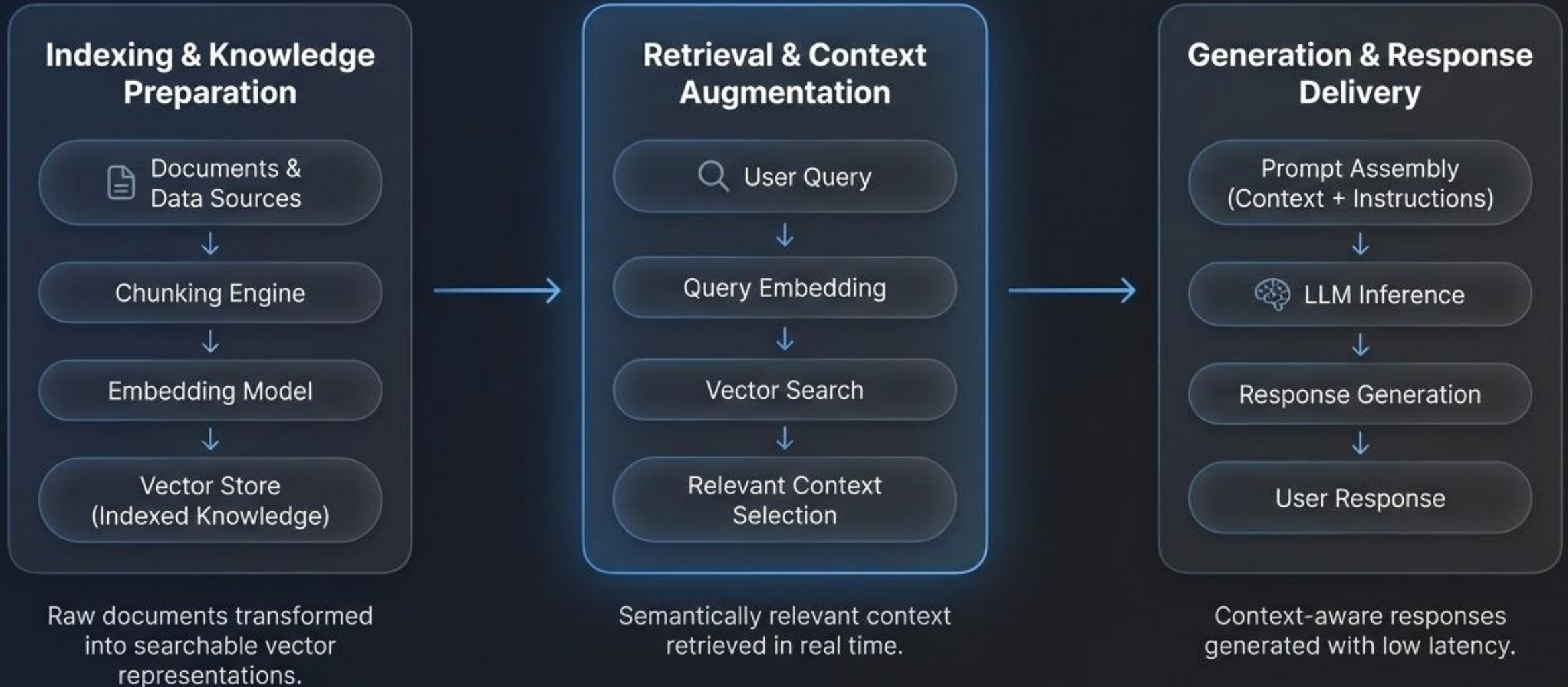
The Cognitive Engine: AI Integration & Adapter Layer

This system integrates Large Language Models (GPT / Gemini) through a specialized AI Adapter Layer. It handles prompt engineering, request batching, and context transformation to convert raw resume and session data into structured, domain-specific interview intelligence.

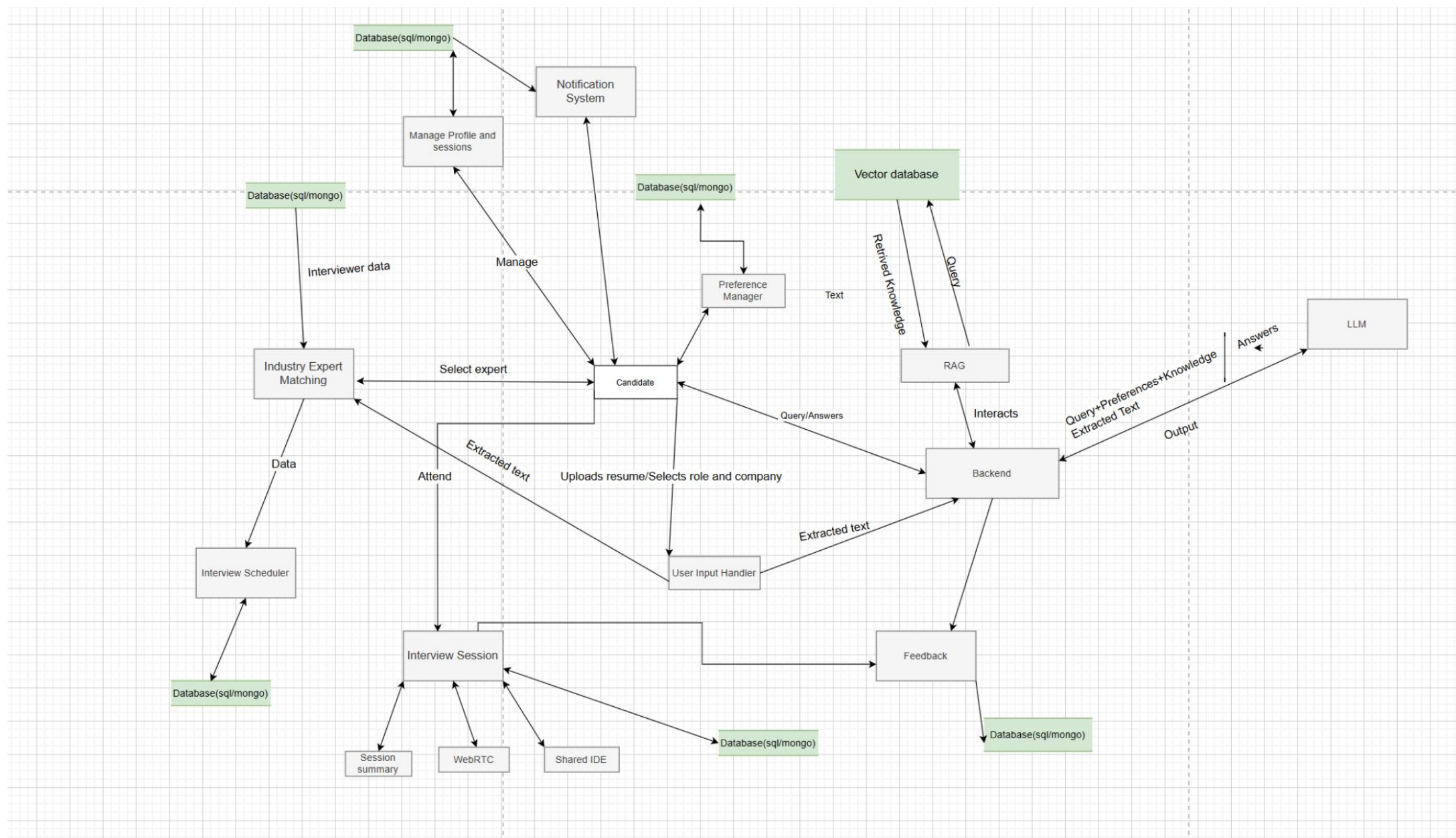


"The system delivers real-time feedback on clarity, confidence, and communication."

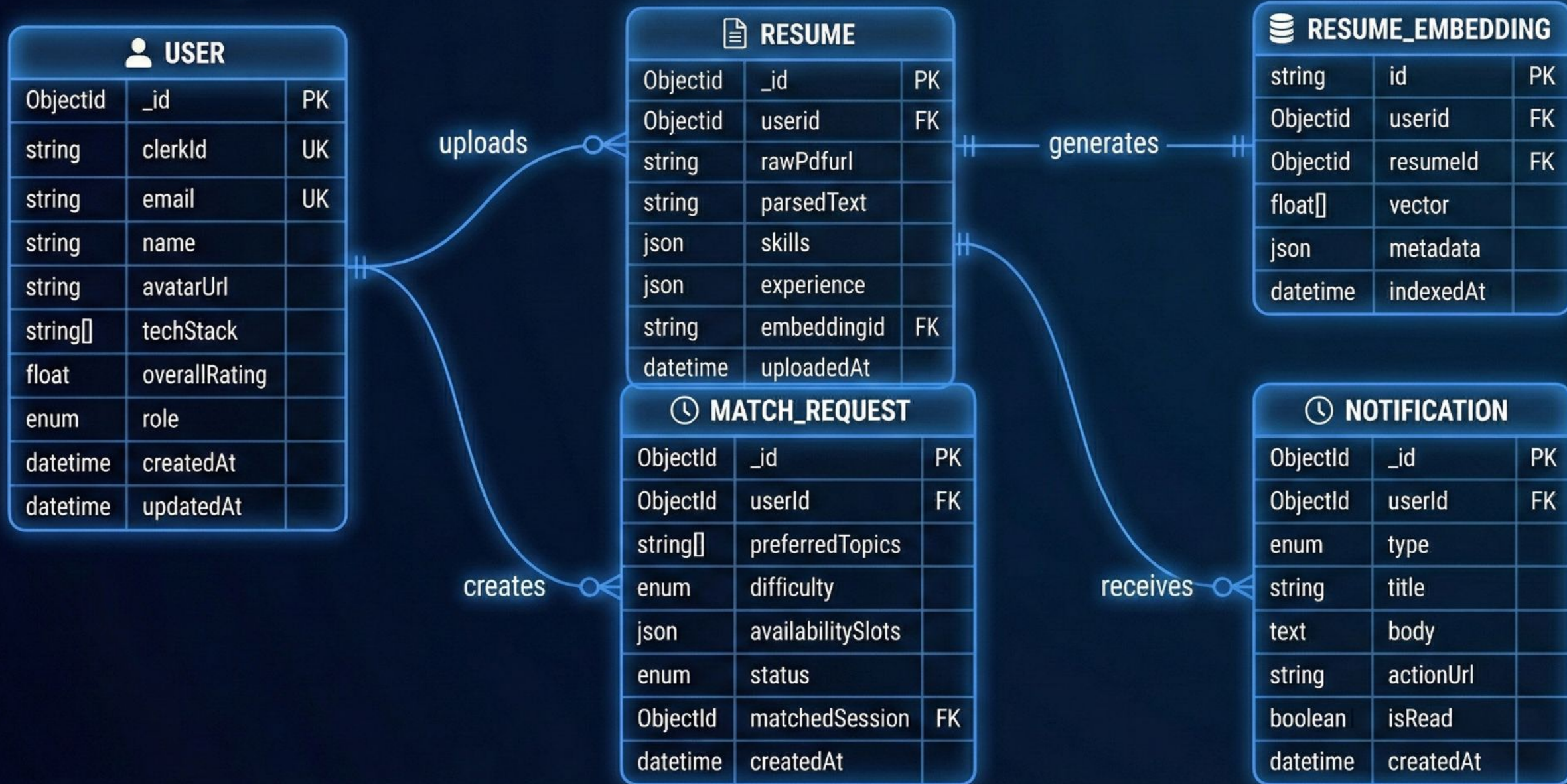
End-to-End System Flow



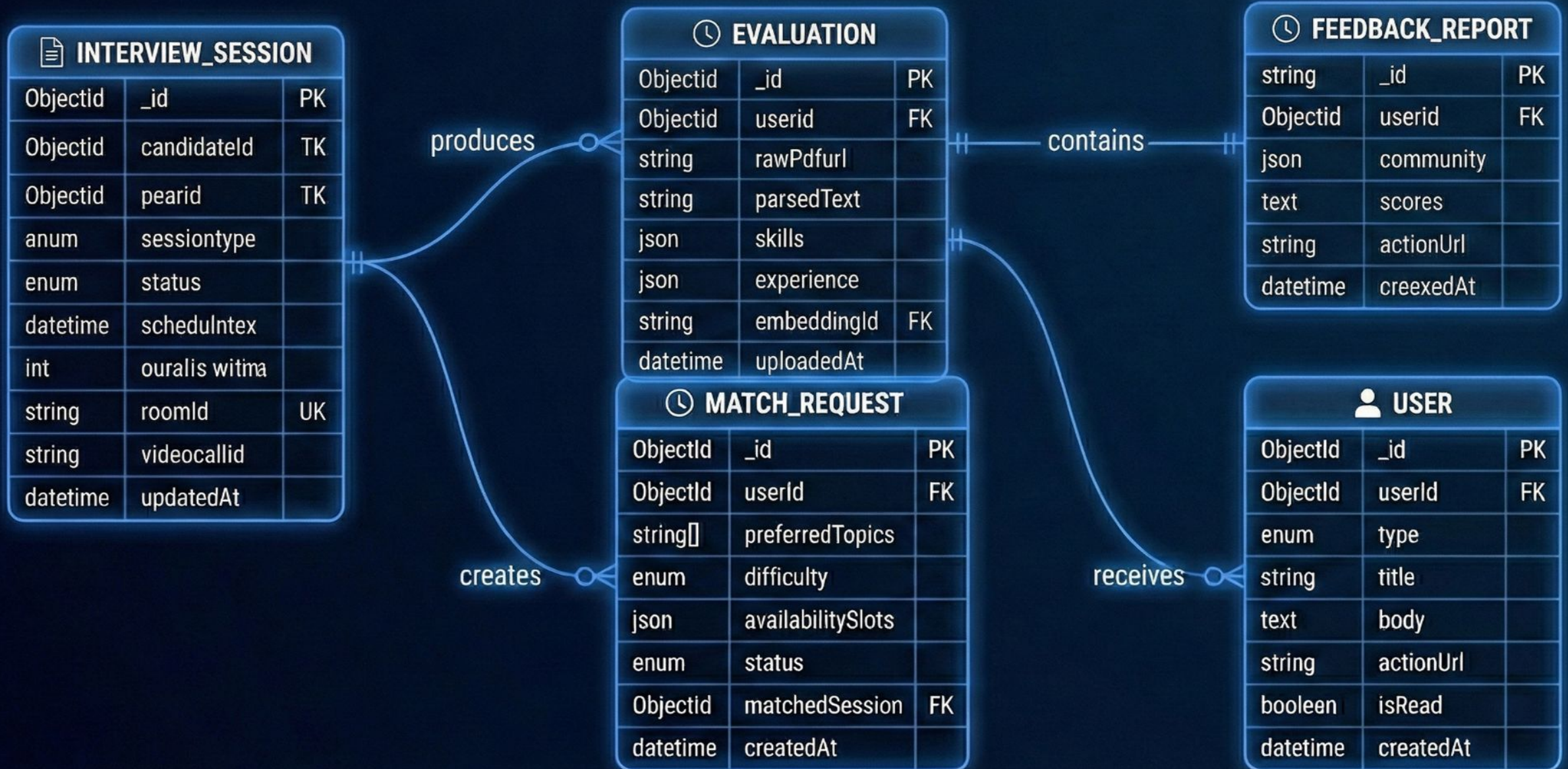
DFD Level 1



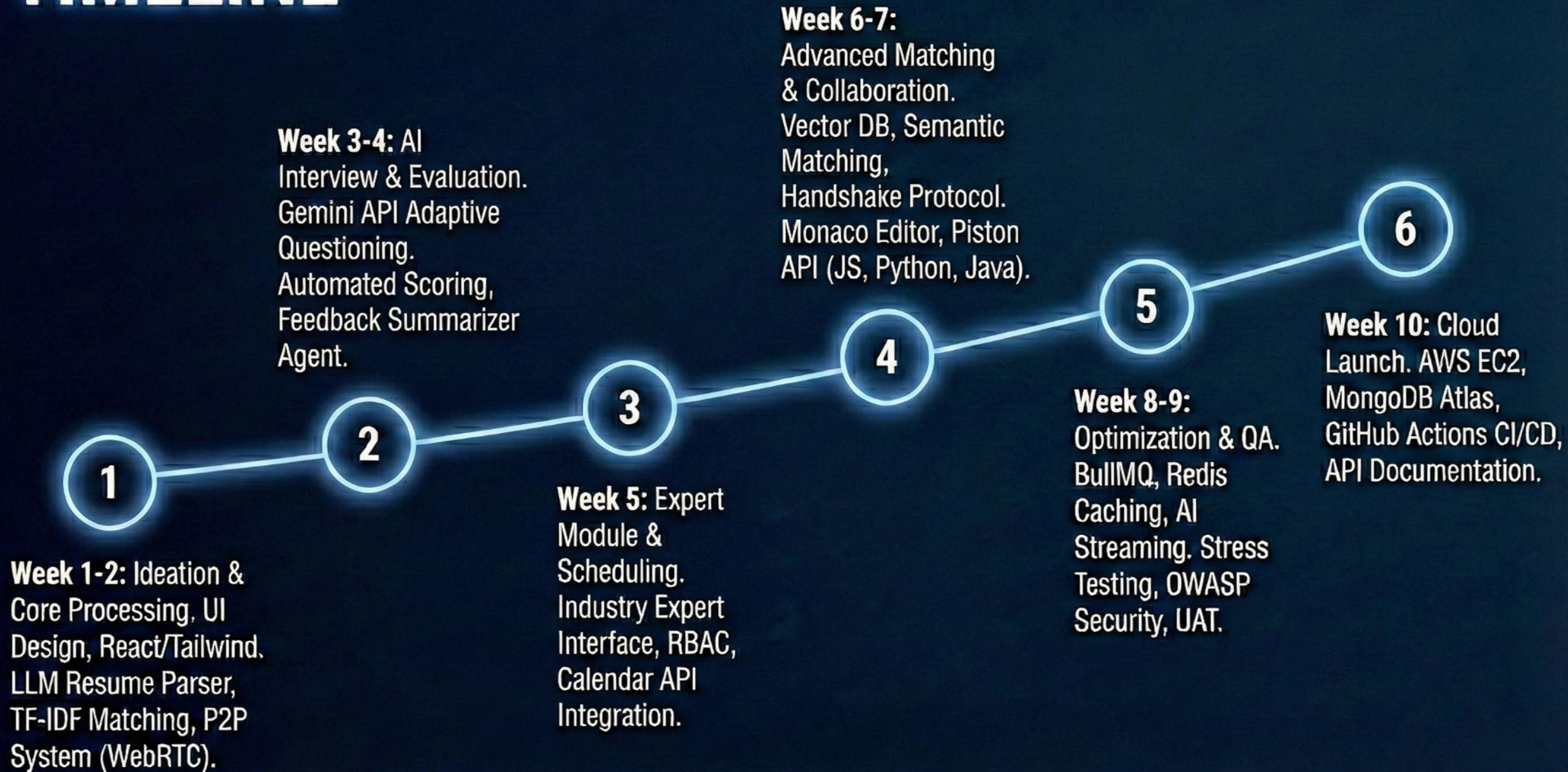
ER Diagram for User, Resume & Matching Domain



ER Diagram for Interview Session, Evaluation & AI Domain



TIMELINE



Our User Interfaces

Talent IQ

Documentation

Sign In

New: AI-Powered Mock Interviews

Master Your Technical Interviews

Collaborate in real-time, get instant AI feedback, and practice with peers. The ultimate platform for developers to level up their career.

Get Started

Watch Demo

10k+
ACTIVE USERS

50k+
SESSIONS COMPLETED

3+
CORE LANGUAGES

Everything you need to succeed

Built for modern engineering teams and aspiring developers.

🔗

Real-time Collaboration

Share code editors instantly with zero latency. Syntax highlighting for JS, Python, and Java with synchronized cursors.

🗣️

AI-Powered Feedback

Receive instant, actionable feedback on your code efficiency, time complexity, and communication skills after every session.

📺

HD Video Conferencing

Crystal clear video and audio built directly into the browser. Simulate the pressure of a real face-to-face technical interview.

Talent IQ

© 2024 Talent IQ. All rights reserved.

Talent IQ

SESSION ID: 8X92-B

LIVE

42:16

Invite

Sarah Miller (Interviewer)

🔇

📺

📄

📞

INTERVIEW STRUCTURE

- Introduction
5 min • Completed
- Coding Challenge
45 min • In Progress
- System Design
15 min • Up Next
- Q&A
5 min • Pending

JavaScript (Node v18)

Saved

Reset

Run Code

```
1 class Solution {
2   /**
3    * @param {number[]} nums
4    * @param {number} target
5    * @return {number[]}
6    */
7   twoSum(nums, target) {
8     const map = new Map();
9
10    for (let i = 0; i < nums.length; i++) {
11      const complement = target - nums[i];
12
13      if (map.has(complement)) {
14        return [map.get(complement), i];
15      }
16
17      map.set(nums[i], i);
18    }
19    return [];
20  }
21 }
```

Console

Test Cases

Problems

→ Running tests...

✓ Test Case 1: [2,7,11,15], target=9 -> Passed (2ms)

✓ Test Case 2: [3,2,4], target=6 -> Passed (1ms)

📢 All tests passed! Ready to submit.

Chat

AI Coach

Today, 10:23 AM

SM

Can you explain your thought process for the hash map approach?

10:24 AM

Sure! I'm using it to store the complement values we need to find.

ME

10:25 AM

Type a message...

Our User Interfaces

 Mock Interview: Invert Binary Tree

MEDIUM

00:14:35

Finish Interview

 AI Interviewer
LISTENING



"Given the root of a binary tree, invert the tree, and return its root."

Transcript

Hints

Resume Context

AI Agent • 10:42 AM

Let's start with a classic. Can you walk me through your thought process for inverting a binary tree?

You • 10:43 AM

Sure. I think I should use a recursive approach. I'll need to swap the left and right children for every node.

AI Agent • 10:43 AM

That sounds like a solid plan. What is the base case for your

HOLD SPACE TO SPEAK

JavaScript



Run Code

```
1
2  var invertTree = function(root) {
3      // Base case: if root is null, return null
4      if (!root) {
5          return null;
6      }
7
8      // Swap the children
9      let temp = root.left;
10     root.left = root.right;
11     root.right = temp;
12
13     // Recursively invert the subtrees
14     invertTree(root.left);
15     invertTree(root.right);
16
17     return root;
18 }
```

Output

Test Cases (3/3)

Console

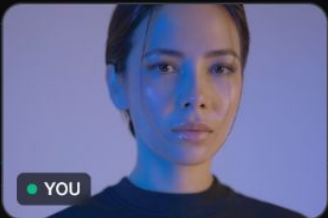
✓ Case 1: Standard Tree

Input: [4,2,7,1,3,6,9]

Exp

✓ Case 2: Empty Tree

 YOU

Our User Interface

